IA para Desarrolladores

Sesión 5 · Transformar — depurar y revisar en equipo

CURSO · 20 HORAS · 10 SESIONES

Agenda de hoy

10'	20'	25'	25'	30'	10'
Repaso: ¿pasan sus pruebas?	Del computador al equipo: ramas, PR y GitHub	Demo: el Depurador	Demo: el Revisor	Práctica: su primer PR revisado	Preguntas, tarea y cierre

Hoy: su mi-reto en GitHub y su primer pull request revisado por un agente y decidido por usted.

Dónde vamos en el ciclo



Lo que hoy se suma al agente del equipo (versión 3)

Depurador

Reproduce una falla con una prueba, corrige la causa raíz con el cambio mínimo y explica qué pasó.

Revisor y /revisar

Revisa una rama contra spec, plan, reglas y políticas. Solo escribe revision.md.

Ramas y pull requests

convenciones.ramas y commits, politicas.revision y la plantilla del pull request.

Repaso: ¿pasan sus pruebas?

```
cd ../mi-reto
python -m pytest -q
python herramientas/verificar_capas.py
```

Escriba en el chat: verde, rojo o no corre.

- Rojo o no corre: hoy lo arregla con el Depurador.
- Verde: hoy avanza un paso del plan en una rama.

Verde no es correcto

Verde dice que lo que probamos funciona. No dice que esté bien. Hoy vamos a ver un error que pasa todas las pruebas, y un cambio que pasa pruebas y capas pero no debería entrar a main.

Del computador al equipo

Ramas, pull requests y GitHub

Primero: ponga al día su repositorio

```
bash proyectos/poner_al_dia.sh ../mi-reto
```

Llega en la versión 3	Claude Code	Copilot
Agente depurador	.claude/agents/depurador.md	.github/agents/depurador.agent.md
Agente revisor	.claude/agents/revisor.md	.github/agents/revisor.agent.md
Skill /revisar y su plantilla	.claude/skills/revisar/	.github/skills/revisar/
Ramas, commits, revisión y depuración	banco.yaml · AGENTS.md	el mismo
Plantilla del pull request	.github/pull_request_template.md	el mismo

Después: cierre y vuelva a abrir claude en mi-reto; en VS Code, Reload Window.

El flujo del equipo



El pull request es donde una persona decide. El agente revisa y sugiere; usted aprueba y fusiona.

Las aprobaciones del ciclo viajan ahí: el commit que pone Estado: aprobado lo hace la persona dueña, dentro de la rama.

Con un hallazgo bloqueante abierto, no se fusiona (politicas.revision).

Las convenciones del banco

Qué	Clave en banco.yaml	Ejemplo
Ramas	convenciones.ramas	correccion/simulador-credito-consumo-endeudamiento
Commits	convenciones.commits	Corrige el veredicto de endeudamiento: compara el valor exacto (RN-05)
Título del PR	convenciones.pull_requests	Como un commit: qué entra y por qué. Corrige el veredicto de endeudamiento (RN-05)
Bloqueante	politicas.revision	Incumple una regla de negocio, una política de datos o seguridad, o rompe pruebas
Importante	politicas.revision	Incumple una convención o deja un criterio sin prueba
Sugerencia	politicas.revision	Mejora opcional
Depurar	politicas.depuracion	Primero reproducir con una prueba; nunca cambiar el valor esperado para que pase

Pregúntele al banco: ¿Qué dice la política de revisión y cuál es el formato de las ramas, los commits y el título del PR?

Su repositorio en GitHub

1

Crearlo

github.com → New repository → mi-reto, Private, sin README ni .gitignore.

2

Conectarlo

git remote add origin https://github.com/<usuario>/mi-reto.git

3

Subirlo

git push -u origin main. La primera vez se abre el navegador para iniciar sesión.

- Privado: .mcp.json guarda la ruta de su Python, con su usuario de Windows.
- Solo datos sintéticos y nada del banco: es su reto del curso.
- Si la red no deja entrar a GitHub: ramas locales y /revisar igual; el PR lo abre en casa.

Un pull request en GitHub, paso a paso

1

Abrir el PR

Después del git push: botón Compare & pull request en la franja amarilla. Si no: Pull requests → New, base main, compare su rama.

2

Título y plantilla

Título como un commit (si GitHub pone el nombre de la rama, cámbielo). La descripción trae la plantilla: caso, rama, etapa, casillas y qué hizo el agente.

3

Create pull request

Queda abierto. En Files changed está el diff: ahí se revisa y se comenta.

4

Merge pull request

Sin bloqueantes: Merge y Confirm merge. Luego, en la terminal: git switch main && git pull.

¿De dónde sale la plantilla?

```
.github/pull_request_template.md
```

- No se configura: GitHub la usa sola si el archivo está en main con ese nombre (en .github/, la raíz o docs/).
- La trae la plantilla del curso; la v3 le agregó rama, revision.md y verificación.
- Para cambiarla: edítela en una rama, PR y merge. Aplica a los PR nuevos.

Cuatro agentes, cuatro permisos

Agente	Lee	Escribe	Terminal	Para qué
arquitecto	todo	solo plan.md	no	Decidir cómo se construye
desarrollador	todo	src/ y tests/	sí	Construir el plan, prueba primero
depurador	todo	src/ y tests/	sí	Reproducir una falla y corregir la causa raíz
revisor	todo	solo revision.md	solo para leer y verificar	Encontrar lo que las pruebas no ven

Ninguno hace commit, push ni merge: eso lo hace usted. El Revisor señala; el dueño del código decide.

¿Por qué agentes y no skills?

	Skill (/spec, /plan-tecnico, /revisar)	Agente (arquitecto, desarrollador, revisor…)
Qué es	Un procedimiento: cómo se hace una tarea	Un rol: quién la hace y con qué límites
Permisos	Los de la sesión que la usa	Los suyos: el arquitecto no toca src/; el revisor solo escribe revision.md
Contexto	Se carga en la conversación actual	Empieza limpio: no «recuerda» las intenciones del autor
Se combinan	Es el método	El arquitecto usa /plan-tecnico; el revisor usa /revisar

Regla práctica: «hágalo así» es una skill; «esto lo hace alguien que solo puede hacer esto» es un agente.

Demo: el Depurador

Primero reproducir, después corregir

Un reporte de una asesora

«Simulé 10.000.000 a 36 meses al 26 %, con ingresos de 2.219.900 y otras cuotas de 500.000. La pantalla dice endeudamiento 40 % y que sí le alcanza, pero la cuenta da 40,04 %: por encima del máximo no debería alcanzar.»

40

pruebas en verde

OK

regla de capas

RN-05

(cuota + otras) / ingresos ≤ 0,40

¿Cuántas pruebas fallan? Ninguna. Ahí empieza el trabajo del Depurador.

Primero reproducir

1

Entender

Qué se esperaba y qué pasó. Lo esperado sale de la spec: RF-03 y RN-05.

2

Reproducir

Una prueba que falla por el motivo del reporte. Si no falla, no está reproducido.

3

Causa raíz

La línea o el supuesto que la produce, con su regla. No el síntoma.

4

Cambio mínimo

Solo lo necesario, en la capa de la causa. Nada de refactorizar de paso.

5

Verificar

Toda la suite en verde y capas en OK. La prueba nueva se queda.

Lo que el Depurador nunca hace: cambiar el valor esperado de una prueba para que pase. Si cree que la prueba está mal, se detiene y pregunta (politicas.depuracion).

Lo que dejó: dos líneas y una prueba

```
# dominio/reglas.py · validar_endeudamiento
-    mostrado = (endeudamiento * 100).quantize(
-        Decimal("0.1"), rounding=ROUND_HALF_UP)
-    if mostrado > maximo * 100:
+    if endeudamiento > maximo:

# tests/test_reglas.py
+def test_endeudamiento_40_04_no_le_alcanza():
+    """Regla: RN-05. 0,4004 supera 0,40."""
+    assert validar_endeudamiento(Decimal("0.4004"),
+        ENDEUDAMIENTO_MAXIMO) is not None
```

Síntoma: 40,04 % «le alcanza».

Causa raíz: compara el porcentaje redondeado para mostrar, no el valor (RN-05).

Por qué pasó: un compañero cambió la regla y ninguna prueba miraba justo por encima del 40 %.

Lo que queda: si alguien vuelve a redondear antes de comparar, la prueba falla (regresión).

Commit: Corrige el veredicto de endeudamiento: compara el valor exacto (RN-05). Luego push, pull request con la plantilla y merge.

Demo: el Revisor

Lo que las pruebas no ven

La rama de un compañero: ¿la fusionamos?

```
git switch funcionalidad/simulador-credito-consumo-casos-referencia
python -m pytest -q                          # 41 passed
python herramientas/verificar_capas.py   # OK
```

Pruebas en verde

Paso 12 del plan: los casos de las oficinas dan la misma cuota y el mismo costo.

Capas en OK

El lector de casos es un adaptador y solo importa el dominio.

¿Entonces?

Antes de fusionar: claude --agent revisor y /revisar simulador-credito-consumo.

Pregunta para el grupo: ¿qué podría estar mal si pasa las pruebas y la regla de capas?

Lo que las pruebas no ven

Severidad	Hallazgo	Regla o política
bloqueante	casos_referencia.csv trae cédulas y nombres con apariencia real	politicas.datos · RF-05 (casos saneados)
bloqueante	print con nombre, cédula e ingresos de cada caso	politicas.datos · politicas.logs
bloqueante	Decimal(float(fila["monto"])): el dinero pasa por float	RN-07 · convenciones.dinero
importante	Los esperados se calcularon con el mismo código que se prueba	politicas.pruebas · regla de pruebas
importante	Paso 12 marcado hecho sin la condición del plan (pregunta 5)	plan.md · paso 12
importante	load_cases: nombre en inglés	politicas.nombres · validar_nombre

Ninguno rompe una prueba. Todos incumplen algo que el equipo ya decidió. Eso es lo que revisa el Revisor: lo que las pruebas no ven.

El Revisor señala, usted decide

1

Guardar la revisión

git add docs/sdlc · commit «Revisión del caso de uso (borrador)» · git push.

2

Comentar el PR

Pegar la sección 7 de revision.md como comentario del pull request.

3

Decidir

Con bloqueantes no se fusiona. El dueño del código decide qué se atiende.

4

Atender

claude --agent desarrollador: «Atiende los hallazgos H-01 y H-02…». En Copilot: botón Atender los hallazgos.

Luego se vuelve a revisar: /revisar agrega una ronda nueva en revision.md. Sin bloqueantes, usted la aprueba y fusiona.

Revisor: ¿agente o subagente?

	Como agente	Como subagente
Cómo se abre	claude --agent revisor · selector de Copilot	En una sesión normal: «Usa el subagente revisor para revisar la rama actual…» o @revisor
Conversa con usted	Sí, mientras trabaja	No: trabaja aparte y devuelve un resumen
Contexto	Toda la sesión es del Revisor	Su propia ventana; no llena la suya
Cuándo conviene	Demos y revisiones que quiere seguir paso a paso	Revisar sin interrumpir lo que está haciendo

El mismo archivo sirve para las dos cosas: Claude Code delega a un subagente cuando la tarea coincide con su description.

Trabajo largo: varios agentes en paralelo

1

Subagentes en una sesión

«Usa en paralelo el subagente revisor sobre la rama del paso 12 y otro sobre la del paso 13». Cada uno en su contexto; devuelven un resumen. Para investigar y revisar, no para editar lo mismo.

2

Sesiones en worktrees

claude --worktree paso-12 y claude --worktree paso-13: cada sesión con su carpeta y su rama. Cada rama termina en un PR revisado.

3

Agent teams

Un orquestador reparte tareas entre varios Claude que se hablan. Experimental y apagado por defecto: todavía no para el equipo.

- Parta por los pasos de plan.md que tocan archivos distintos; dos agentes sobre reglas.py a la vez es mal paralelo.
- El límite es su revisión: cada agente deja una rama que alguien lee y decide. Dos o tres a la vez; cada uno gasta tokens.

¿Y si la convención se cumpliera sola?

Nivel	Cómo	Qué logra
Plantilla de commit	Archivo .gitmessage y git config commit.template .gitmessage	Ayuda: aparece al hacer git commit sin -m. No obliga.
Hooks de Git	commit-msg y pre-push en .githooks/; git config core.hooksPath .githooks	Obliga en el computador: rechaza el commit o el push si no cumple banco.yaml
Revisión en GitHub	Una GitHub Action en cada PR (sesión 8)	Obliga para todos: marca en rojo la rama o el título del PR
Rulesets de GitHub	Patrones de nombre de rama y de mensaje de commit	En la organización del banco; en una cuenta personal gratuita, no

Hoy la convención está en banco.yaml y la citan los agentes. Para ramas no hay plantilla en Git: se valida con un hook o en el PR. En la sesión 6 lo veremos: una regla aconseja, un hook obliga.

Manos a la obra: su primer pull request revisado

8'

GitHub

Repositorio privado mi-reto,
remote y git push -u origin main.

12'

Rama y agente

Rojo: correccion/… y depurador.
Verde: funcionalidad/…-paso-<n>
y desarrollador.

7'

Revisión

claude --agent revisor
/revisar <caso>
commit de revision.md

3'

Pull request

git push -u origin <rama>
Compare & pull request
No fusionar con bloqueantes.

Copilot: elija Depurador, Desarrollador o Revisor en el selector; al terminar, el botón Revisar los cambios pasa al Revisor.

Los comandos de Git de hoy

Para	Comando
Ver en qué rama está	git branch --show-current
Crear una rama y pasarse a ella	git switch -c <tipo>/<caso>-<descripcion>
Ver qué cambió frente a main	git diff main...HEAD --stat
Subir la rama (la primera vez)	git push -u origin <rama>
Volver a main y traer lo fusionado	git switch main && git pull
Ver el historial como árbol	git log --oneline --all --graph

Para casa: fusionar sin bloqueantes

1

Atender

Los hallazgos bloqueantes e importantes con el desarrollador (o explique en el PR por qué no).

2

Volver a revisar

Commit, git push y /revisar <caso>: el Revisor agrega una ronda nueva.

3

Fusionar

Sin bloqueantes: revision.md en aprobado con su nombre, commit, push y Merge pull request.

4

Entregar

El enlace de su PR fusionado (o una captura) por el canal, antes de la sesión 6.

Verde no es correcto. Lo que entra a main lo revisa alguien, y lo decide una persona.

Para la sesión 6

1

Probar mejor

El agente Verificador y la skill /pruebas: casos desde los criterios de la spec, bordes y cobertura.

2

Datos sintéticos

Una herramienta nueva del banco genera datos de prueba con la forma del catálogo, sin datos reales.

3

Hooks

Las pruebas y la regla de capas se corren solas cada vez que el agente edita: ya no dependen de acordarse.

Una regla aconseja; un hook obliga.